DEPLOYING APPLICATIONS WITH DOCKER

DEPLOYING STATIC WEBSTITE USING DOCKER

1. vim Dockerfile

- FROM ubuntu:latest
- LABEL author="dev mayur"
- RUN apt update && \
  apt install apache2 unzip -y
- ADD https://templatemo.com/download/templatemo_623_novapay /var/www/html/ templatemo_623_novapay.zip
- RUN rm -rf /var/www/html/index.html && \
  unzip /var/www/html/templatemo_623_novapay.zip -d /var/www/html/ && \
  mv /var/www/html/templatemo_623_novapay/* /var/www/html/
- EXPOSE 80
- CMD ["apache2ctl", "-D", "FOREGROUND"]

2. docker build -t apache2 .
3. docker images
4. docker run -d -p 8081:80 –name apache2 –rm apache2
   <http://public_ip:8081>
5. docker tag apache2 mayurpandit25/apache2:latest
6. docker push mayurpandit25/apache2:latest

DEPLOYING MONOLITHIC APPLICATION USING DOCKER

{Project_url: https://github.com/mayurpandit25/studentapp-ui.git}

1.Create A RDS Database (MySQL)

1. username ---> admin
2. password ---> admin123
3. database ----> studentapp

2.Create A Instance

1. Install docker
2. git clone https://github.com/mayurpandit25/studentapp-ui.git
3. cd studentapp-ui

4. vim Dockerfile

- FROM ubuntu:22.04
- RUN apt update -y && \
  apt install -y curl tar openjdk-11-jdk
- ADD https://dlcdn.apache.org/tomcat/tomcat-9/v9.0.118/bin/apache-tomcat-9.0.118.tar.gz /opt/
- WORKDIR /opt/
- RUN tar -xzf /opt/apache-tomcat-9.0.118.tar.gz && \
  rm /opt/apache-tomcat-9.0.118.tar.gz
- COPY context.xml /opt/apache-tomcat-9.0.118/conf/
- ADD https://s3-us-west-2.amazonaws.com/studentapi-cit/student.war /opt/apache-tomcat-9.0.118/webapps/student.war
- ADD https://s3-us-west-2.amazonaws.com/studentapi-cit/mysql-connector.jar /opt/apache-tomcat-9.0.118/lib/mysql-connector.jar
- EXPOSE 8080
- RUN chmod +x /opt/apache-tomcat-9.0.118/bin/catalina.sh
- CMD ["/opt/apache-tomcat-9.0.118/bin/catalina.sh","run"]

5. Edit In context.xml

1. username --> admin
2. password --> admin123
3. Endpoint ---> rds_database endpoint

6. Install Mysql-Client

1. apt update
2. apt install mysql-client -y
3. systemctl start mysql
4. mysql -u admin -h <rds database endpoint> -padmin123
   USE studentapp;
   CREATE TABLE IF NOT EXISTS students (
   student_id INT NOT NULL AUTO_INCREMENT,
   student_name VARCHAR(100),
   student_addr VARCHAR(100),
   student_age VARCHAR(3),
   student_qual VARCHAR(20),
   student_percent VARCHAR(10),
   student_year_passed VARCHAR(10),

```
    PRIMARY KEY (student_id)
);
```

7. **Build Image & Run container**
   1. docker build -t studentapp .
   2. docker run -d -p 8080:8080 –name student-app studentapp:latest

8. **Check Application**
   <http://public_ip:8080/student>

DEPLOYING THREE-TIER APPLICATIONS WITH DOCKER

{ Project_url: https://github.com/mayurpandit25/EasyCRUD-Updated-df.git }

1.Create A RDS Database

1. username -------> admin
2. password --------> admin123
3. database ---------> student_db

2.Create A Instance

1. name --------------> webserver
2. machine ----------> ubuntu
3. instance_type ---> m7iflex.large
4. create a instance

3.Access The Instance

1. Install docker
2. git clone https://github.com/mayurpandit25/EasyCRUD-Updated-df.git
3. cd EasyCRUD-Updated-df

4.BACKEND

1. cd backend
2. cd src/main/resources/
3. vim application.properties
   1. username ----------> admin
   2. password -----------> admin123
   3. database_url ------> < paste the endpoint from rds database >

```
root@ip-172-31-46-14:~/EasyCRUD-Updated-df/backend/src/main/resources# cat application.properties
server.port=8080

spring.datasource.url=jdbc:mariadb://database-1.cd26siuswor4.ap-south-1.rds.amazonaws.com:3306/student_db?sslMode=trust
spring.datasource.username=admin
spring.datasource.password=mayur123

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

4. cd backend
5. vim Dockerfile
   - FROM maven:3.8.3-openjdk-17
   - COPY . /opt/
   - WORKDIR /opt
   - RUN mvn clean package
   - WORKDIR target/
   - EXPOSE 8080
   - ENTRYPOINT ["java","-jar"]
   - CMD ["student-registration-backend-0.0.1-SNAPSHOT.jar"]
6. docker build -t backend .

4.FRONTEND

1. cd frontend
2. vim .env
   VITE_API_URL=http://13.235.246.18:8080/api
3. vim Dockerfile
   - FROM node:24-alpine
   - COPY . /opt/
   - WORKDIR /opt
   - RUN npm install && npm run build
   - RUN apk update && apk add apache2
   - RUN rm -rf /var/www/localhost/htdocs/*
   - RUN cp -rf dist/* /var/www/localhost/htdocs
   - EXPOSE 80
   - ENTRYPOINT ["httpd","-D","FOREGROUND"]
4. docker build -t frontend .

5.RUN THE CONTAINERS

1. docker run -d -p 8080:8080 –name backend –rm backend
2. docker run -d -p 80:80 –name frontend –rm frontend

6.CHECK ON WEB

1. http://public_ip:8080/api ----------> Backend works
2. http://public_ip ----------> Frontend works

DEPLOYING THREE-TIER APPLICATION ON LOCAL

1.Install & Setup Mysql

1. apt install mysql-server -y
2. systemctl start mysql
3. mysql
    1. CREATE DATABASE student_db;
    2. CREATE USER 'admin'@'localhost' IDENTIFIED BY 'Admin@123';
    3. GRANT ALL PRIVILEGES ON student_db.* TO 'admin'@'localhost';
    4. FLUSH PRIVILEGES;

2.Clone The Application

1. git clone https://github.com/mayurpandit25/EasyCRUD-Updated-df.git
2. cd EasyCRUD-Updated-df

3. Set Up Backend

1. apt install openjdk-17-jdk -y
2. apt install maven -y
3. cd backend
4. cd src/main/resource/
5. vim application.properties

```
server.port=8080
spring.datasource.url=jdbc:mariadb://localhost:3306/student_db?sslMode=trust
spring.datasource.username=admin
spring.datasource.password=Admin@123
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

6. cd backend
7. mvn clean package
8. ls target
9. java -jar student-registration-backend-0.0.1-SNAPSHOT.jar

4.Set Up Frontend

1. curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash -
2. sudo apt install nodejs -y
3. apt install apache2 -y
4. systemctl start apache2
5. cd frontend
6. npm install
7. vim .env
   VITE_API_URL="http://localhost:8080/api"
8. npm run build
9. cp -rf dist/* /var/www/html/
10. systemctl restart apache2

5.Check on Web

1. < http://localhost:8080/api >
2. < http://localhost >

DEPLOY THREE-TIER APPLICATION LOCALLY USING DOCKER

1.Install Docker

1. apt update
2. apt install docker.io -y
3. systemctl start docker
4. systemctl enable docker

2.Clone Project

1. git clone https://github.com/mayurpandit25/EasyCRUD-Updated-df.git
2. cd EasyCRUD-Updated-df

3.Create A Docker Network

1. docker network create student-network

4.Run Mariadb Container

docker run -d --name mysql-db --network student-network -e MYSQL_ROOT_PASSWORD=root123 -e MYSQL_DATABASE=student_db -e MYSQL_USER=admin -e MYSQL_PASSWORD=admin123 -p 3306:3306 mariadb:latest

5.Configure Backend

1. cd backend/src/main/resources
2. vim application.properties

server.port=8080
spring.datasource.url=jdbc:mariadb://mysql-db:3306/student_db?sslMode=trust
spring.datasource.username=admin
spring.datasource.password=admin123
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

3.Create A Backend Dockerfile

1. cd backend
2. vim Dockerfile
   - FROM maven:3.8.3-openjdk-17
   - COPY . /opt/
   - WORKDIR /opt
   - RUN mvn clean package -DskipTests
   - WORKDIR /opt/target
   - EXPOSE 8080
   - ENTRYPOINT ["java","-jar"]
   - CMD ["student-registration-backend-0.0.1-SNAPSHOT.jar"]
3. docker build -t backend .
4. docker run -d --name backend --network student-network -p 8080:8080 Backend

4.Configure Frontend

1. vim .env
   VITE_API_URL=http://localhost:8080/api

5.Create A Frontend Dockerfile

1.vim Dockerfile

- FROM node:24-alpine
- COPY . /opt/
- WORKDIR /opt
- RUN npm install && npm run build

- RUN apk update && apk add apache2
- RUN rm -rf /var/www/localhost/htdocs/*
- RUN cp -rf dist/* /var/www/localhost/htdocs/
- EXPOSE 80
- ENTRYPOINT ["httpd","-D","FOREGROUND"]

3. docker build -t frontend .
4. docker run -d --name frontend --network student-network -p 80:80 frontend

5.Check Containers

2. database
3. backend
4. frontend

6.Access Application

1. < http://localhost:8080/api >
2. < http://localhost >

7.To Access Mariadb Container

1. docker exec -it mysql_db bash
2. mariadb -u admin -padmin123
